LAPORAN PRAKTIKUM PEMROGRAMAN BERBASIS OBJEK

Minggu Ke-3

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Nurfiah S.ST, M.Kom

Asisten Pratikum: Siti Fadhilah Rahmi



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI ..... **i**

KATA PENGANTAR ..... **ii**

BAB I PENDAHULUAN **1**

1.1 Latar Belakang **1**

1.2 Tujuan Praktikum **1**

1.3 Manfaat Praktikum **1**

BAB II PEMBAHASAN **2**

2.1 Penguncian Entitas Transaksi **2**

2.2 Penguncian Entitas Rekening dan Penambahan PIN **2**

BAB III TUGAS **4**

3.1 Meminta User Menginputkan PIN **4**

3.2 Validasi PIN Sebelum Tarik Tunai dan Mencetak Mutasi **4**

BAB IV KESIMPULAN **6**

4.1 Ringkasan Hasil Praktikum **6**

DAFTAR PUSTAKA ..... **iii**

KATA PENGANTAR

Puji syukur kehadirat Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Pemrograman Berbasis Objek.

Ucapan terima kasih disampaikan kepada Bapak Nurfiah S.ST, M.Kom. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam Pemrograman Berorientasi Objek (PBO), perangkat lunak dimodelkan berdasarkan entitas yang ada di dunia nyata. Dua pilar paling mendasar dalam pemodelan ini adalah Class dan Object. Class berfungsi sebagai cetak biru (blueprint) dari sebuah objek yang mendefinisikan struktur data (atribut atau state) serta sekumpulan operasi (metode atau behavior) yang dapat dilakukan pada data tersebut. Class itu sendiri tidak memakan ruang memori untuk menyimpan nilai data, melainkan hanya berfungsi sebagai tipe data referensi. Di sisi lain, Object adalah instansiasi konkret dari sebuah class yang dialokasikan di dalam memori komputer, di mana setiap objek memiliki state atau kondisi yang saling independen. Selain itu, PBO juga menggunakan Constructor, yaitu blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat untuk menginisialisasi state awal dari objek tersebut.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

- Mendefinisikan class sebagai model dari entitas dunia nyata.
- Mendefinisikan object dan memahami alokasi memori independen.
- Menggunakan constructor untuk inisialisasi awal state pada objek.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

- Meningkatkan pemahaman mengenai manajemen memori dinamis di Java.
- Meningkatkan fondasi logika untuk membangun struktur data dan arsitektur program yang lebih kompleks di masa depan.

BAB 2

PEMBAHASAN

2.1 Penguncian Entitas Transaksi

```
public class Transaksi {
	private String idTransaksi;
	private String jenis;
	private double nominal;

	// constructor
	public Transaksi(String id, String jenis, double nominal)
	{
		this.idTransaksi = id;
		this.jenis = jenis;
		this.nominal = nominal;
	}

	// getter only
	public String getIdTransaksi() { return idTransaksi; }
	public String getJenis() { return jenis; }
	public double getNominal() { return nominal; }
```

Pada tahap ini, pengamanan data dilakukan dengan menerapkan konsep enkapsulasi di class Transaksi. Seluruh atribut kelas, yaitu idTransaksi, jenis, dan nominal, diubah menjadi private agar detail implementasi internal tidak dapat dimanipulasi secara langsung dari luar kelas. Karena objek transaksi bersifat immutable atau tidak boleh diubah setelah dibuat, maka sistem tidak menyediakan method Setter, melainkan hanya menyediakan method Getter

2.2 Penguncian Entitas Rekening dan Penambahan PIN

```
package praktikumMinggu1;

import java.util.ArrayList;

public class Rekening {
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin;

    // Implementasi asosiasi
    ArrayList<Transaksi> riwayatTransaksi;
```

```
    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // validasi pin
        while (true) {
        if (pinAwal.matches("\\d{6}")) {
             this.pin = pinAwal;
             break;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit dan harus
angka!");
            return;
            }

        }

        //inisialisasi arraylist didalam constructor
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat dengan saldo Rp" + saldo);
    }

    // getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    // method otentikasi internal (validasi enkapsulasi)
    public boolean otentikasi(String inputPin) {
      return this.pin.equals(inputPin);
    }
```

Pengamanan diterapkan pada class Rekening dengan menggunakan access modifier private pada atribut sensitif, termasuk saldo dan koleksi riwayatTransaksi. Selain itu, menambahkan lapisan keamanan baru berupa atribut PIN yang juga bersifat private. Konstruktor dimodifikasi untuk menerima parameter PIN awal saat pembuatan rekening, yang disertai dengan validasi logika untuk memastikan panjang PIN harus berjumlah 6 digit. Untuk memvalidasi keamanan, sebuah method otentikasi(String inputPin) ditambahkan untuk membandingkan masukan pengguna dengan PIN yang tersimpan.

BAB 3

TUGAS

3.1 Meminta User Menginputkan PIN

```
                    System.out.print("Masukkan PIN Awal: ");
                    String pin = input.nextLine();


                    Rekening akunBaru = new Rekening(no, nama, saldo, pin);
                    daftarRekening.add(akunBaru);
                    akunAktif = akunBaru;
                    break;
```

Modifikasi pada rekening untuk menambahkan pengamanan PIN baru. Sebelum program menginstansiasi objek Rekening, sistem meminta pengguna untuk menginputkan PIN awal. Input tersebut kemudian diteruskan sebagai argumen tambahan pada saat pemanggilan constructor pembentukan akun, sehingga objek baru langsung terikat dengan kredensial keamanan pengguna.

3.2 Validasi PIN Sebelum Tarik Tunai dan Mencetak Mutasi

```
case 3:
                    if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum
memiliki nomor rekening aktif!");
                    } else {
                          System.out.print("Masukkan PIN Anda: ");
                          pin = input.nextLine();

                          if (akunAktif.otentikasi(pin)) {
                              System.out.print("Masukkan nominal tarik
tunai: ");

                              double tarik = input.nextDouble();
                              akunAktif.tarikTunai(tarik);
                          } else {
                              System.out.println("Akses Ditolak: PIN yang
Anda masukkan salah!");
                          }
                    }
```

```
case 6:
                    // Tugas 2, Pemanggilan method cetakMutasi()
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum membuka
rekening!");
```

```
                    } else {
                          System.out.print("Masukkan PIN Anda: ");
                          pin = input.nextLine();

                          if (akunAktif.otentikasi(pin)) {
                              akunAktif.cetakMutasi();
                          } else {
                              System.out.println("Akses Ditolak: PIN yang
Anda masukkan salah!");
                          }
                    }
                    break;
```

Sesuai dengan aturan keamanan baru, setiap eksekusi fungsi yang memengaruhi atau menampilkan state sensitif, seperti Tarik Tunai (Menu 3) dan Cetak Mutasi (Menu 6), harus dilindungi oleh otentikasi. Sistem terlebih dahulu meminta input PIN dari pengguna, lalu memanggil fungsi akunAktif.otentikasi(pinYangDiinput). Fungsi ini mengembalikan nilai boolean; jika bernilai true, transaksi dilanjutkan, namun jika bernilai false, program akan membatalkan transaksi dan mencetak peringatan "Akses Ditolak: PIN yang Anda masukkan salah!"

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa penerapan enkapsulasi krusial dalam menjaga integritas data dalam rekayasa perangkat lunak. Penggunaan access modifiers bertipe private secara efektif melindungi atribut-atribut rentan dari modifikasi ilegal yang dapat merusak logika bisnis sistem. Akses terhadap data dikontrol sepenuhnya melalui method Getter untuk pembacaan dan metode otentikasi internal untuk validasi keamanan, sehingga operasional perbankan hanya dapat dijalankan apabila pengguna memiliki PIN yang valid.

DAFTAR PUSTAKA

[1] Nurfiah. Modul Praktikum PBO - Java: Praktikum 3 Keamanan & Integritas Data (Enkapsulasi). [Diakses: 24 September 2026].